

Code. Build. Ship.

A modern property rental platform — built end-to-end.

Problems

PROBLEM 01

Outdated listings.

Renters scroll through stale ads with vague addresses, missing photos and no way to verify if the place is still available.

PROBLEM 02

No tooling for owners.

Posting a property is a manual chore — photos, geocoding and updates are spread across spreadsheets and messengers.

PROBLEM 03

Fragmented communication.

Renters and owners juggle phone, WhatsApp and email. There is no shared place to chat, book a viewing and keep history.

Methodology

01

Domain-Driven Design

Business rules live in pure Java domain objects — User, Listing, Address, Photo, Conversation — free of framework noise.

02

Hexagonal architecture

Inbound REST controllers and outbound JPA / geocoding adapters plug into the core via clean ports. Swappable, testable.

03

REST + OpenAPI

Versioned JSON API documented with OpenAPI. Stateless, JWT-secured, consumed by the Angular SPA and ready for future clients.

04

DevOps & GitOps

Every push on main is mirrored to GitLab, built into a Docker image, pushed to ACR and rolled out to Kubernetes.

Technology stack

Front-end

- Angular 20 + Signals
- Reactive Forms
- RxJS
- PrimeNG
- TypeScript
- Nginx (static serving)

Back-end

- Spring Boot 4
- Spring Data JPA
- PostgreSQL + Liquibase
- MapStruct
- Keycloak (OAuth2 / JWT)
- OpenStreetMap geocoding

DevOps

- Docker
- Kubernetes
- GitLab CI/CD
- GitHub Actions mirror
- Azure Container Registry
- Ingress + TLS

Access on any device

Mobile-first responsive layout — desktop, tablet, phone share the same component tree.



Desktop screenshot



Mobile screenshot

What it does

01

Multi-step wizard

Listing creation in six guided steps with per-step validation, draft state and inline server errors.

02

Geocoded address

Every address is verified and geocoded via Nominatim — coordinates persisted on the listing.

03

Photo gallery

Up to 20 photos per listing with cover image, drag-reorder and per-photo alt text.

04

In-app messaging

Renters and owners chat per-listing. Read state, soft delete and listing context inline.

05

Viewing requests

Booking flow with date, time and notes. Owner can confirm, decline or propose a new slot.

06

Owner dashboard

Stats, listing statuses (active / paused / archived) and bulk actions in one place.

From push to production

GitHub → GitLab mirror → CI build → registry → Kubernetes rollout.



Architecture / pipeline diagram

Future steps

01

Payments & deposits

Stripe-powered booking deposits with refundable holds and owner payouts.

02

Map-based search

Cluster-based map view with on-map filters and bounding-box queries on the API.

03

Notifications

Real-time WebSocket inbox plus email digests for new messages and viewing requests.

04

Owner analytics

Per-listing views, lead funnel, response time and revenue projections.

What I've learned

1

Ship a full-stack feature.

From a Liquibase migration to an Angular signal — owning every layer of one feature is a different skill than writing isolated code.

2

Boundaries pay off.

Keeping the domain free of Spring / JPA made writing tests and swapping the geocoder painless.

3

Async UX is the hard part.

Optimistic state, server validation errors and Angular change detection (NG0100) only show up when you wire real network calls.

4

Pipelines beat manual deploys.

Once GitHub → GitLab → K8s was wired, iteration speed jumped. The pipeline is the product.

5

Sweat the small things.

Toast wording, button disabled states, photo order — the invisible details decide whether the app feels finished.

Thank you.



Questions, code review, follow-up — all welcome.